

Semântica Distribucional

Aula 15 – Introdução ao Processamento de Linguagem Natural (PLN)

Eduardo Corrêa Gonçalves

11/05/2026

Sumário

Introdução

Semântica Distribucional – Visão Geral

Hipótese Distribucional

Modelos Semânticos Distribucionais

Vetores Semânticos

Matriz de Coocorrência

Matriz Termo-Documento

Matriz Termo-Contexto

Similaridade entre Vetores Semânticos

Similaridade de Cosseno

Introdução (1/3)

- Leia a frase abaixo:

“Ser feliz sem motivo é a forma mais autêntica de felicidade.”

- É relativamente fácil para **nós, seres humanos**:
 - Extrair informações da frase a partir de uma simples leitura.
 - Reconhecer o sentido das palavras e o significado do texto formado pela junção de todas as palavras.

Introdução (2/3)

“Ser feliz sem motivo é a forma mais autêntica de felicidade.”

- Entretanto para **eles**, os **algoritmos** e **métodos estatísticos**:
 - Não há como entender palavras.
 - Ao invés, eles precisam de uma representação numérica do texto a ser processado
 - Apenas assim, conseguirão realizar suas operações.

Introdução (3/3)

- É aí que entra a **Semântica Distribucional (SD)**
 - Principal abordagem para a representação do **significado** lexical na PLN.
- Princípio básico – “**palavras como vetores**”:
 - Palavras são representadas por meio de **vetores** de **valores reais**.
 - São denominados **vetores semânticos**.
 - Eles **codificam o significado** das palavras a partir de sua distribuição em textos.

Semântica Distribucional (1/7)

- A SD é ancorada na **Hipótese Distribucional** ,
 - “palavras que têm um **contexto linguístico semelhante**, tendem a ter **significado similar** ou aproximado”
 - Teoria formulada na década de 1950 por linguistas como **J. R. Firth** e **Zellig S. Harris**.
 - Eles perceberam que:
 - Palavras com **significado relacionado** (ex.: “**quente**” e “**abafado**”) ...
 - ... tendem a ocorrer em um **mesmo contexto** (ex.: perto de certas palavras como “**clima**”, “**dia**”, “**local**”)

Semântica Distribucional (2/7)

- **Exemplo** (Parrish, 2018):
 - “Estava muito frio ontem.”
 - “Ficará muito quente amanhã”
 - “Vai fazer muito calor na terça?”
- De acordo com a Hipótese Distribucional:
 - As palavras “frio”, “quente” e “calor” devem possuir algum tipo de relação.
 - Uma vez que ocorrem em um **contexto similar**.
 - Nesse caso, entre a palavra “muito” e uma palavra que indica um **dia em particular**.

Semântica Distribucional (3/7)

- Exemplo:
 - “Estava muito frio ontem.”
 - “Ficará muito quente amanhã”
 - “Vai fazer muito calor na terça?”
- De maneira análoga:
 - “ontem”, “amanhã” e “terça” devem estar relacionadas.
 - Já que ocorrem em um contexto de uma palavra que indica temperatura !

Semântica Distribucional (4/7)

"You shall know a word by the company it keeps"

J. R. Firth, 1957

- Para a Hipótese Distribucional, o significado de uma palavra é baseado em uma grande lista de todos os contextos em que ela ocorre.
- Palavras que compartilham contextos possuem alguma relação semântica.

Semântica Distribucional (5/7)

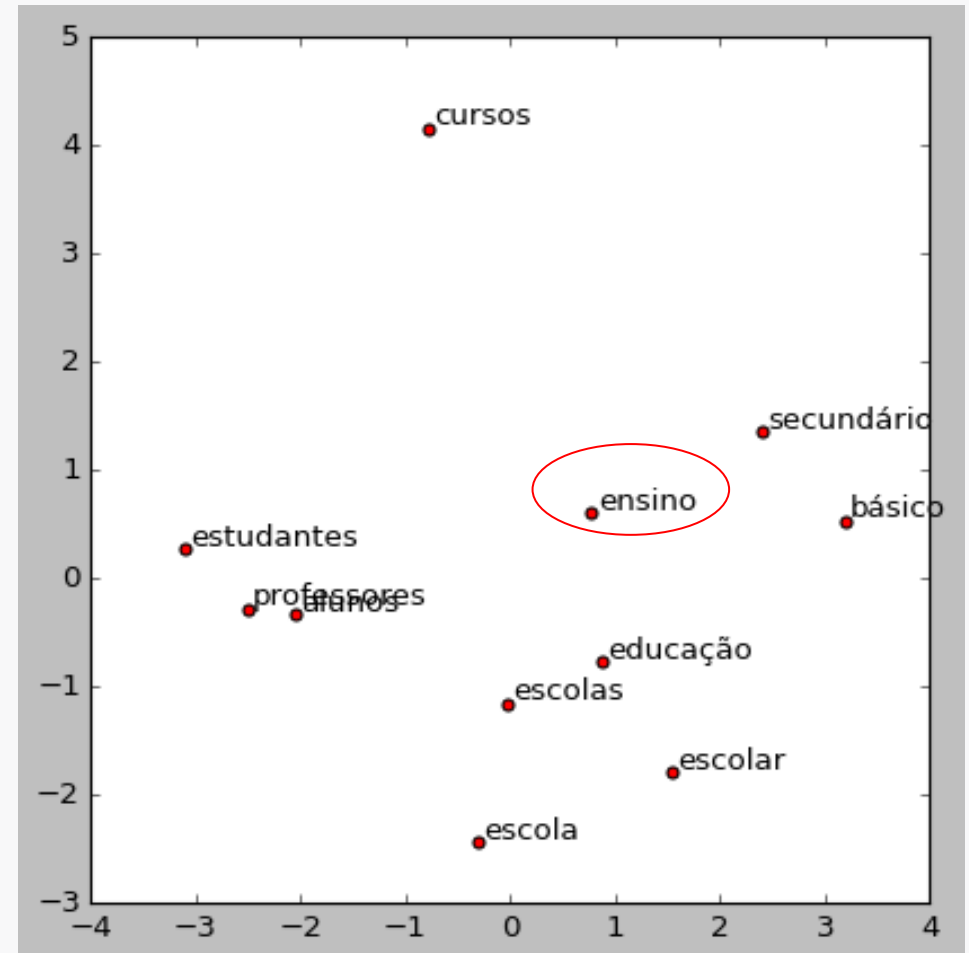
- **Modelos Semânticos Distribucionais (MSD)**
 - São modelos que empregam a Hipótese Distribucional para fazer o computador “entender” a semântica das palavras.
 - Têm por objetivo produzir **vetores** de propósito geral que capturam o **significado das palavras**.
 - São denominados **vetores semânticos**
 - Os MSD produzem uma representação vetorial que retrata o significado de uma palavra a partir da distribuição das palavras que formam o seu contexto.

Semântica Distribucional (6/7)

- Modelos Semânticos Distribucionais (MSD)

- Exemplo** (Seno et al. 2026): espaço vetorial semântico de “**ensino**”.

- Note que palavras como “**educação**” e “**escolar**” compartilham esse mesmo espaço semântico.
- Cada ponto do espaço multidimensional representa a conotação de uma palavra.



Semântica Distribucional (7/7)

- **Vetores Semânticos**

- A representação vetorial semântica é o padrão de representação mais comum em PLN.
- Ela pode retratar **vários aspectos** do **significado** das palavras.

- **Exemplo** (Seno et al., 2026):

- similaridade → “**comércio**” × “**negócios**”
- sentimento → “**fenomenal**” × “**estúpido**”
- associação → “**futebol**” × “**bola**”

Matriz de Coocorrência (1/10)

- Vetores semânticos podem ser criados usando uma **matriz de coocorrência**
 - Basicamente, matriz que retrata a **frequência de coocorrência** das **palavras**.
 - 2 tipos mais comuns:
 - Matriz Termo-Documento
 - Matriz Termo-Contexto

Matriz de Coocorrência (2/10)

- (i) Matriz Termo-Documento (*Term-Document Matrix - TDM*)
 - Nesta matriz, o espaço vetorial é formado por uma coleção de **documentos** que **representam os vetores** ou pontos nesse espaço.
 - **Ex.:** TDM p/ 5 termos (palavras) extraídas de 3 documentos (Seno et al., 2024):

	DOC ₁ : energias renováveis	DOC ₂ : escassez de energia	DOC ₃ : veículos elétricos
energia	32	36	9
eólico	15	15	1
renovável	6	8	4
etanol	1	1	22
hidrogênio	0	0	16

Matriz de Coocorrência (3/10)

- Na **TDM** → cada vetor semântico tem dimensão $|V|$, onde $|V|$ é o vocabulário (num. de palavras distintas).
 - Linha = palavra;
 - Coluna = documento;
 - Célula = freq. da palavra i no doc j
- Cada **documento** (coluna) da matriz é um **vetor semântico**
 - Ela captura algo sobre a semântica de documentos, mas não das palavras.
 - Veja que o **DOC₁** é mais parecido com o **DOC₂** do que com o **DOC₃**

	DOC₁: energias renováveis	DOC₂: escassez de energia	DOC₃: veículos elétricos
energia	32	36	9
eólico	15	15	1
renovável	6	8	4
etanol	1	1	22
hidrogênio	0	0	16

Matriz de Coocorrência (4/10)

- **TF-IDF** → As células da TDM também podem armazenar o valor do **TF-IDF** da palavra i no doc j , como no exemplo abaixo.

	DOC₁: energias renováveis	DOC₂: escassez de energia	DOC₃: veículos elétricos
energia	0.246	0.371	0.030
eólico	0.118	0.120	0.001
renovável	0.030	0.024	0.004
etanol	0.002	0.008	0.199
hidrogênio	0.000	0.000	0.098

- **Observações:**
 - Veja que a TDM é a transposta da DTM (apresentada na aula sobre TF-IDF)
 - Além do TF-IDF, outras medidas podem ser usadas, como a **Pointwise Mutual Information – PMI** (informações em Jurafsky e Martin (2026))

Matriz de Coocorrência (5/10)

- Qual é a grande DESVANTAGEM da TDM ?
 - Em aplicações reais, os **vocabulários** podem ter **centenas de milhares de palavras**.
 - E o número de **documentos** pode ser enorme (até **milhões** !!!)
 - Além disso, a maioria dos documentos possuirá poucas palavras em relação ao total de palavras do vocabulário.
- Consequência:** serão produzidos **vetores enormes** e muito **esparsos** (terão uma grande quantidade de elementos com valor 0).

	DOC ₁ : energias renováveis	DOC ₂ : escassez de energia	DOC ₃ : veículos elétricos
energia	0.246	0.371	0.030
eólico	0.118	0.120	0.001
renovável	0.030	0.024	0.004
etanol	0.002	0.008	0.199
hidrogênio	0.000	0.000	0.098

Matriz de Coocorrência (6/10)

- (ii) Matriz Termo-Contexto

- Nesta matriz, o espaço vetorial é formado por uma coleção de **palavras**, que **representam os vetores** nesse espaço.
 - Cada **célula** representa o número de vezes que a **palavra da linha** (alvo) e a **palavra da coluna** (contexto) **coocorrem em algum contexto** no corpus.
 - Sendo assim, a dimensionalidade é $|V| \times |V|$
 - O objetivo é que os vetores representem o significado das palavras.
 - Também é conhecida por matriz palavra-palavra ou termo-termo.

Matriz de Coocorrência (7/10)

- Matriz Termo-Contexto

- Mas **o que é contexto**???
 - Normalmente, um **número de palavras à esquerda e à direita da palavra-alvo** (linha).
 - Por exemplo, 3 palavras à esquerda e 3 palavras à direita.
 - Dessa forma, cada célula representa o número de ocorrência da palavra da coluna (contexto) em uma janela de três palavras em torno da palavra-alvo.
- Ex.:** matriz termo-contexto do corpus da Wikipedia*:

	aadvark	aargh	...	computer	data	pie
cherry	0	0		2	8	442
strawberry	0	0		0	0	60
digital	0	0		1670	1683	5
information	0	0		3325	3982	5

* Exemplo retirado de Jurafsky e Martin (2026)

Matriz de Coocorrência (8/10)

- **Exemplo de Matriz Termo-Contexto – Corpus da Wikipedia**
 - Linha = representa cada palavra-alvo
 - Coluna = representa o contexto
 - Célula = indica o número de vezes que a palavra-alvo (linha) coocorreu em um dado contexto (coluna)
- Por exemplo, um **contexto de tamanho 5** → significa 5 palavras à esquerda e 5 à direita da palavra-alvo.

	aadvark	aargh	...	computer	data	pie
cherry	0	0		2	8	442
strawberry	0	0		0	0	60
digital	0	0		1670	1683	5
information	0	0		3325	3982	5

Matriz de Coocorrência (9/10)

- Exemplo de Matriz Termo-Contexto – Corpus da Wikipedia
 - Cada **palavra-alvo** representa um **vetor**.
 - Assim temos os vetores das palavras “**cherry**”, “**strawberry**”, “**digital**” e “**information**”. **Veja que:**
 - Os vetores de “**digital**” e “**information**” são razoavelmente similares e bem diferentes dos vetores de “**cherry**” e “**strawberry**”
 - Da mesma forma, os vetores “**cherry**” e “**strawberry**” são mais parecidos entre si do que com os outros 2 vetores;

	aadvark	aargh	...	computer	data	pie
cherry	0	0		2	8	442
strawberry	0	0		0	0	60
digital	0	0		1670	1683	5
information	0	0		3325	3982	5

Matriz de Coocorrência (10/10)

- Qual é a grande DESVANTAGEM da Matriz Termo-Contexto ?
 - Em aplicações reais, os **vocabulários** têm **milhares de palavras**.
 - E a maioria das palavras não ocorrerá no contexto de uma palavra-alvo.
 - Neste exemplo “**aadvark**” e “**aargh**” não coocorrem com “**cherry**”, “**strawberry**”, “**digital**” e “**information**”.
 - Consequência:** teremos novamente **vetores esparsos**.

	aadvark	aargh	...	computer	data	pie
cherry	0	0		2	8	442
strawberry	0	0		0	0	60
digital	0	0		1670	1683	5
information	0	0		3325	3982	5

Embeddings (1/3)

- O que são embeddings?
 - Vetores das palavras como os gerados por Matrizes Termo-Contexto são chamados de **embeddings**.

cherry	0	0	...	2	1	442
--------	---	---	-----	---	---	-----

strawberry	0	0	...	1	0	507
------------	---	---	-----	---	---	-----

digital	0	0	...	1670	1683	5
---------	---	---	-----	------	------	---

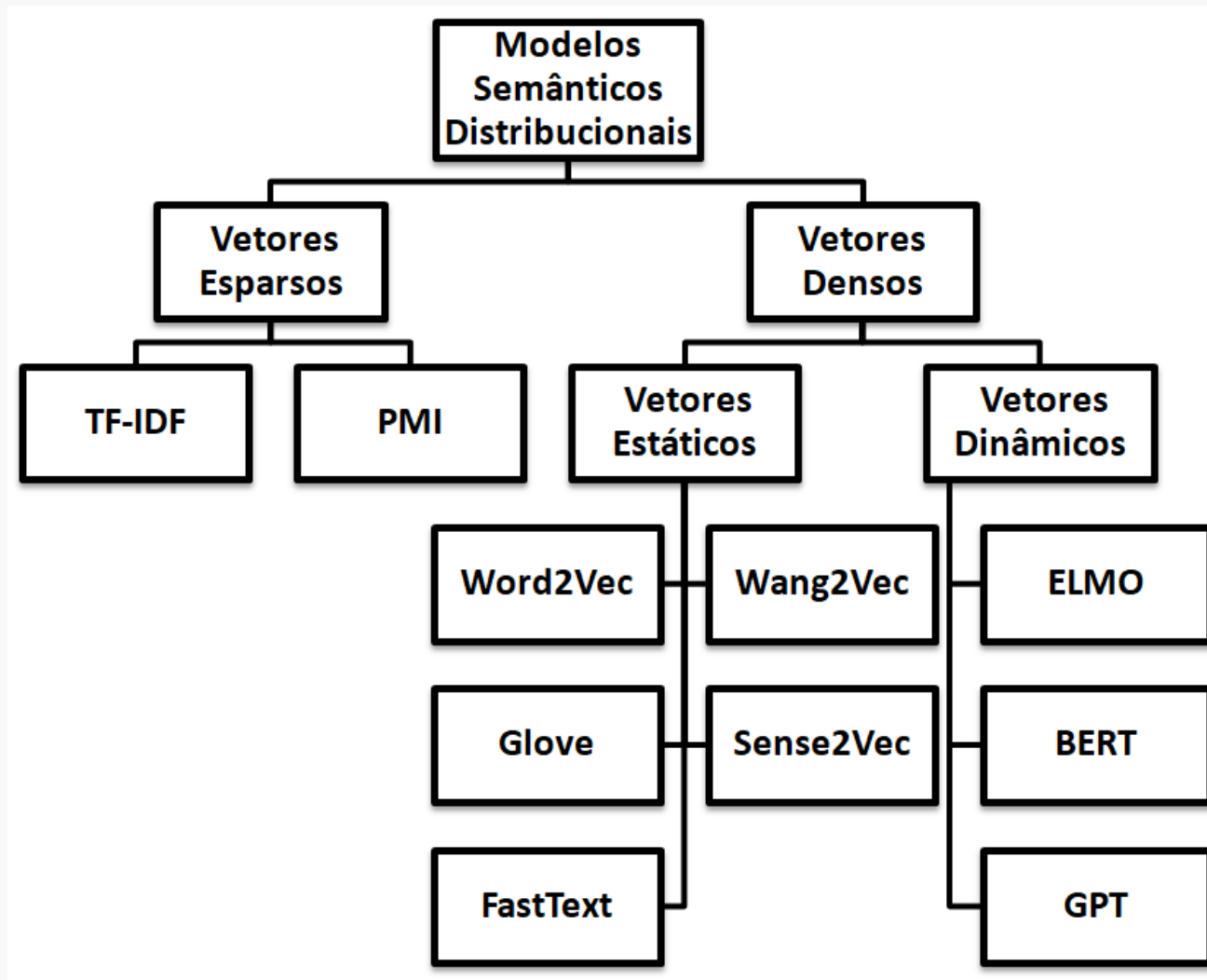
information	0	0	...	3982	3982	5
-------------	---	---	-----	------	------	---

Embeddings (2/3)

- O que são embeddings?
 - Nesta aula, abordamos modelos semânticos distribucionais baseados apenas em **vetores esparsos**...
 - ... porém os embeddings que são realmente **usados na prática** são os **vetores densos** ...
 - Word2Vec
 - Glove
 - FastText
 - Wang2Vec
 - Sense2Vec
 - ELMO
 - BERT
 - GPT
 - ... Em especial, os **vetores densos dinâmicos** como ELMO, BERT e GPT, que serão cobertos na parte final de nosso curso.

Embeddings (3/3)

- Taxonomia de Modelos Semânticos Distribucionais



Similaridade entre Embeddings (1/10)

- Como medir a similaridade entre 2 embeddings v_1 e v_2 ?
 - Normalmente, utiliza-se a medida do **cosseno** do ângulo entre v_1 e v_2
 - Como veremos nos próximos slides, o cosseno é **baseado no produto escalar** dos vetores, algo muito comum na PLN.
- **Produto Escalar** (*dot product* ou *inner product*):
 - Dados dois vetores de n posições v_1 e v_2 , o produto escalar $v_1 \cdot v_2$ entre eles é dado por:

$$v_1 \cdot v_2 = v_{1[1]}v_{2[1]} + v_{1[2]}v_{2[2]} + \dots + v_{1[n]}v_{2[n]}$$

Similaridade entre Embeddings (2/10)

- Por que o Produto Escalar?
 - Ele atua como uma medida de similaridade porque tende a ser alto apenas quando v_1 e v_2 têm valores grandes nas mesmas posições.

$$v_1 \cdot v_2 = v_{1[1]}v_{2[1]} + v_{1[2]}v_{2[2]} + \dots + v_{1[n]}v_{2[n]}$$

- Além disso, **vetores ortogonais** (com zeros em dimensões diferentes), terão um produto escalar de 0.
 - Isso representa bem a forte dissimilaridade entre eles.
- Entretanto, o produto escalar possui alguns problemas...

Similaridade entre Embeddings (3/10)

- **Problema 1:** o Produto Escalar viola a definição de função de similaridade
 - Conforme visto em aulas anteriores, **função de similaridade** entre esses dois objetos s e t , uma é uma função $S : (s, t) \rightarrow [0;1]$ que satisfaz **três propriedades**:
 1. $S(s, t) = 1$ se $s = t$;
 2. $S(s, t) \approx 1$ quando s é muito parecido com t ;
 3. $S(s, t) \approx 0$ quando s é muito diferente de t .
 - Entretanto, o produto escalar pode ser um valor entre $-\infty$ e $+\infty$.

Similaridade entre Embeddings (4/10)

- **Problema 2:** o Produto Escalar favorece vetores com muitos valores grandes.
 - Embeddings de **palavras mais frequentes**, como “fui” ou “você” tendem a possuir **valores grandes** em muitas posições...
 - ... isso porque essas palavras tendem a coocorrer com diversas outras.

fui	150	50	...	480	235	42
você	302	89	...	240	389	361

- Por outro lado, embeddings de **palavras infrequentes**, como “vós” tendem a possuir **zero** ou **valores pequenos** em muitas posições...
 - ... isso porque essas palavras coocorrem com poucas outras

vós	0	2	...	0	0	1
-----	---	---	-----	---	---	---

Similaridade entre Embeddings (5/10)

- **Problema 2:** o Produto Escalar favorece vetores com muitos valores grandes.
 - Dessa forma, “fui” e “você” terão produto escalar com valor superior a “vós” e “você”
 - Embora “vós” e “você” sejam palavras mais similares

fui	150	50	...	480	235	42
você	302	89	...	240	389	361
vós	0	2	...	0	0	1

- Isso é um **problema**!
 - Queremos uma medida de similaridade que funcione bem para palavras frequentes e infrequentes.
 - *É aí que o cosseno entra para resolver !!!!*

Similaridade entre Embeddings (6/10)

- **Cosseno**

- Vetores de palavras frequentes (com valores grandes em mais posições) possuem **comprimento** (ou **norma** ou **módulo**) maior.
- A norma de um vetor **v**, representada por **$\|v\|$** é computada por:

$$\|v\| = \sqrt{v_{[1]}^2 + v_{[2]}^2 + \dots + v_{[n]}^2}$$

- Então a solução é simples: **dividir** o valor do **produto escalar pelo comprimento** de cada vetor !!!
- Isso resulta exatamente no **cosseno do ângulo** entre os vetores:

$$v_1 \cdot v_2 = \|v_1\| \cdot \|v_2\| \cdot \cos(v_1, v_2) \Rightarrow \cos(v_1, v_2) = \frac{v_1 \cdot v_2}{\|v_1\| \cdot \|v_2\|}$$

Similaridade entre Embeddings (7/10)

- **Cosseno dos Vetores**

$$\cos(v_1, v_2) = \frac{v_1 \cdot v_2}{\|v_1\| \cdot \|v_2\|}$$

- Numerador = produto escalar
- Denominador = produto das normas (comprimentos)

- **Como funciona?**

- Quanto menor o ângulo entre os vetores, mais similares eles são.
 - O que interessa é o ângulo, não o tamanho dos vetores.
 - Funciona bem para vetores densos ou esparsos.
- Para embeddings que armazenam frequências, resultará sempre em um valor entre 0 e 1.
 - $\text{Cos}(s, t) \approx 1$, se os vetores são muito similares ($\cos(0^\circ) = 1$)
 - $\text{Cos}(s, t) \approx 0$, se os vetores são muito dissimilares ($\cos(90^\circ) = 0$)

Similaridade entre Embeddings (8/10)

- **Cosseno** – o que importa é o **ângulo entre os vetores** e não a magnitude

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

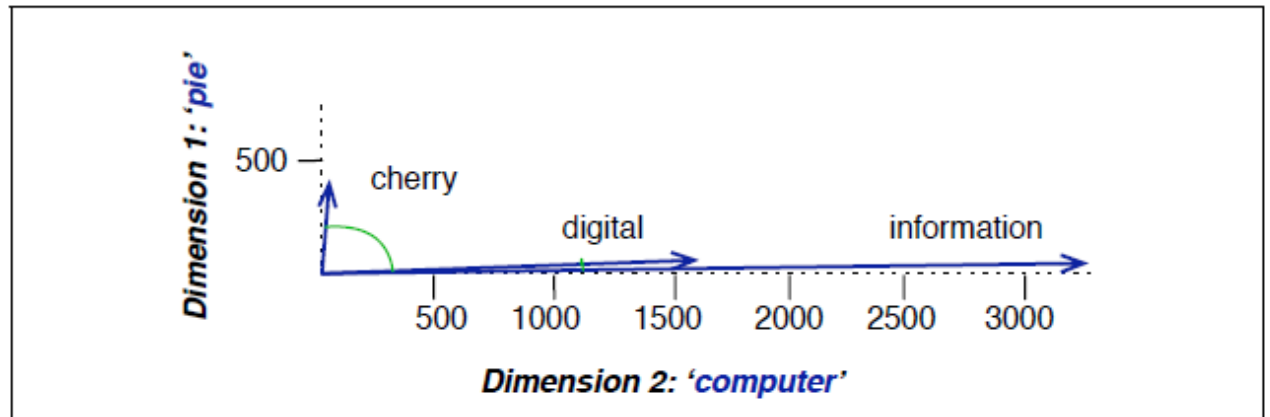


Figure 6.8 A (rough) graphical demonstration of cosine similarity, showing vectors for three words (*cherry*, *digital*, and *information*) in the two dimensional space defined by counts of the words *computer* and *pie* nearby. The figure doesn't show the cosine, but it highlights the angles; note that the angle between *digital* and *information* is smaller than the angle between *cherry* and *information*. When two vectors are more similar, the cosine is larger but the angle is smaller; the cosine has its maximum (1) when the angle between two vectors is smallest (0°); the cosine of all other angles is less than 1.

$$\cos(\text{cherry}, \text{information}) = \frac{442 * 5 + 8 * 3982 + 2 * 3325}{\sqrt{442^2 + 8^2 + 2^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .018$$

$$\cos(\text{digital}, \text{information}) = \frac{5 * 5 + 1683 * 3982 + 1670 * 3325}{\sqrt{5^2 + 1683^2 + 1670^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .996$$

Similaridade entre Embeddings (9/10)

- Cálculo do Cosseno – Exemplo em Python padrão

v_1				
2	0	1	2	0

v_2				
2	1	0	1	1

$$\cos(v_1, v_2) = \frac{v_1 \cdot v_2}{\|v_1\| \cdot \|v_2\|}$$

$$\cos(v_1, v_2) = \frac{(2 \times 2) + (0 \times 1) + (1 \times 0) + (2 \times 1) + (0 \times 1)}{\sqrt{2^2 + 0^2 + 1^2 + 2^2 + 0^2} \times \sqrt{2^2 + 1^2 + 0^2 + 1^2 + 1^2}} = \frac{6}{\sqrt{9} + \sqrt{7}} = \mathbf{0.76}$$

Similaridade entre Embeddings (10/10)

- Cálculo do Cosseno – Exemplo em Python padrão

```
import math

def f_prod_escalar(v1, v2):
    return sum( [i*j for i, j in zip(v1, v2)] )

def f_norma(v):
    return math.sqrt( sum( [i**2 for i in v] ) )

def f_cosseno(v1, v2):
    norma_v1 = f_norma(v1); norma_v2 = f_norma(v2)
    if (norma_v1 > 0) and (norma_v2 > 0):
        return f_prod_escalar(v1, v2) / (norma_v1 * norma_v2)
    else: return 0.0

# as features são os bigramas de caracteres: "an", "ás", "ba", "na", "ná"
v1 = [2, 0, 1, 2, 0] # bigramas de "banana" -> {ba, an, na, an, na}
v2 = [2, 1, 0, 1, 1] # bigramas de "ananás" -> {an, na, an, ná, ás}

print('\nCosseno(v1, v2): ', f_cosseno(v1, v2))
```

Cosseno(v1, v2): 0.7559289460184544

Referências

- JURAFSKY, D.; MARTIN, J. H. Embeddings. *In*: JURAFSKY, D.; MARTIN, J. H. **Speech and Language Processing**. 3. ed. [s.l.] : [s.n.], 2026. cap 5, p. 1-10. Disponível em: <https://web.stanford.edu/~jurafsky/slp3/>. Acesso em: 07 mai. 2026.
- PARRISH, A. **Understanding word vectors**. Github.com, 2018. Disponível em: < <https://gist.github.com/aparrish/2f562e3737544cf29aaf1af30362f469> >. Acesso em: 05 dez. 2024.
- SENO, E. R. M. *et al*. Semântica distribucional. *In*: CASELI, H. M.; NUNES, M. G. V. **Processamento de Linguagem Natural: Conceitos, Técnicas e Aplicações em Português**. 4. ed. [s.l.]: BPLN, 2026. cap 10 (vol 1), p. 1-7. Disponível em: < <https://brasileiraspln.ufscar.br/livro-pln-4ed-vol1/significado/cap-semantica-distribucional/cap-semantica-distribucional.html> >. Acesso em: 07 mai. 2026.